

Test Plan 2 — Product Public Sanity (No-Login Dev Calculator)

Project	product
Specs	tests/product/api/public-app-sanity.spec.ts , tests/product/api/token-sanity.spec.ts
Default filter	none (all specs in product run)
Target	Dev calculator — QA_TARGET_ENV (default dev dev1.app.organize.organuz.com)
Page object	tests/product/support/ProductAppPage.ts
Cases	8 (5 public-app + 3 token)
Skips	Only on a genuine dev outage (gateway/gate down, no UI token)
Skill	product-public-sanity

Scope

Public, unauthenticated checks against the live dev product calculator. They open the app through the shared password gate (PRODUCT_PLATFORM_PASSWORD), confirm the calculator shell loads for a signed-out visitor, and verify the front-end's backend token is well-formed and transmitted securely (in the body, over HTTPS, never in the URL). No sign-in, no personal data.

Preconditions

- Live dev app reachable; PRODUCT_PLATFORM_PASSWORD set (local .env / CI secret).
- TokenInterceptor can observe the UI's backend calls to extract the token.

Gating (sanctioned skip)

- **AppUnavailableError / OtpUnavailableError** → the whole group skips with a clear "dev app unavailable (not a product bug)" reason.

- **token-sanity** additionally skips when no UI token can be extracted. A token that IS observed but malformed or drifted from config **fails** (real regression).

Cases

`public-app-sanity.spec.ts` — "Public dev calculator sanity" (`@product @sanity`)

ID	CASE	ASSERTS
PUB-01	The dev calculator shell loads after unlocking the password gate	App shell renders once the gate is unlocked
PUB-02	A fresh visitor is signed out on the public calculator	No authenticated session for a first-time visitor
PUB-03	The login entry point is available to a signed-out visitor	A visible path to sign in exists
PUB-04	The app is served over HTTPS on the configured dev host	Origin is HTTPS on the expected dev host
PUB-05	The calculator issues a public backend call carrying the token on load	On load the app makes a backend call bearing the token

`token-sanity.spec.ts` — "Product API sanity via extracted UI token (dev)" (`@product @api @sanity`)

ID	CASE	ASSERTS	TAGS
TOK-01	The UI transmits a well-formed token to the backend	Extracted token has the expected shape	<code>@critical</code>
TOK-02	The extracted token matches the bundle public token in config	UI token equals the public token in <code>config.json</code>	—
TOK-03	Every UI backend call sends the token in the body over HTTPS, never the URL	Token only in request body over HTTPS; never a query param	<code>@security</code>

Run

```
QA_TARGET_ENV=dev npx playwright test --project=product tests/product/api
```

Notes

- Never use `waitForLoadState('networkidle')` — the map iframe keeps the network busy; rely on `domcontentloaded` / `expect` auto-waiting.
- Environmental outages are typed errors (`tests/product/support/errors.ts`); the spec edge decides skip-vs-fail, not the page object.